

Binary Search Tree(이진 탐색 트리), UnionFind(유니온 파인드)

■ 소유자	 암암코딩
■ 태그	
■ 생성 일시	@2024년 6월 8일 오후 6:14

<https://youtu.be/mk8-6FmrquY?feature=shared>

https://youtu.be/z22HzKj_Hcs?feature=shared

이진 탐색 트리(Binary Search Tree)

이진 탐색 트리란?

- 이진 탐색의 속성이 이진 트리에 적용된 특별한 형태의 이진 트리

이진 탐색이란?

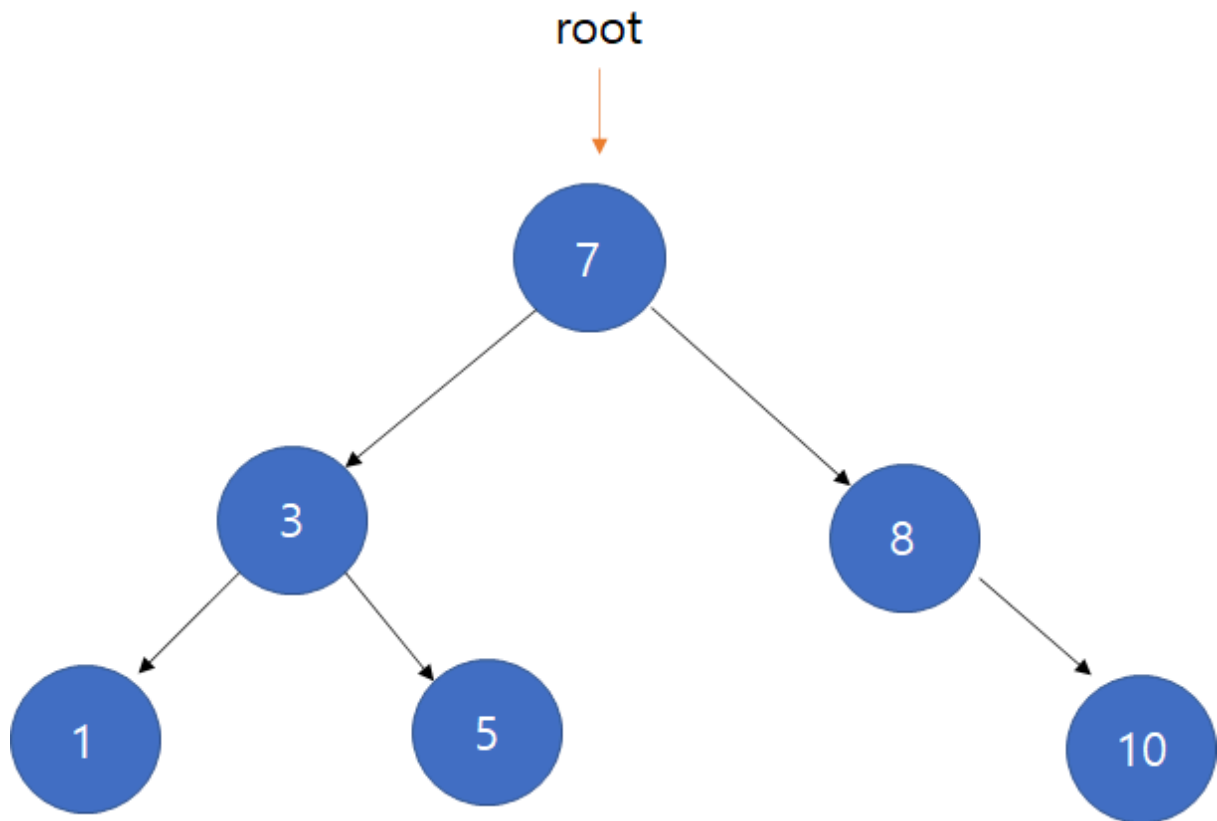
- 이진 탐색 알고리즘이란 **정렬된** 데이터 중에서 특정한 값을 찾기 위한 탐색 알고리즘 중 하나다.
- 이진 탐색 알고리즘은 오름차순으로 정렬된 정수의 배열을 같은 크기의 두 부분 배열로 나눈 후, 두 부분 중 탐색이 필요한 부분에서만 탐색하도록 탐색 범위를 제한하여 원하는 값을 찾는 알고리즘입니다.
 - 배열의 중간에 내가 찾고자 하는 값이 있는지 확인한다.
 - 중간값이 내가 찾고자 하는 값이 아닐 경우,
 - 오름차순으로 정렬된 배열에서 중간값보다 큰 값인지 작은 값인지 판단한다.
 - 찾고자 하는 값이 중간값보다 작은 값일 경우,

- 배열의 맨 앞부터 중간값 전까지의 범위를 탐색 범위로 잡고 탐색을 반복 수행한다.
4. 찾고자 하는 값이 중간값보다 큰 값일 경우,
- 배열의 중간값의 다음 값부터 맨 마지막까지를 탐색 범위로 잡고 탐색을 반복 수행한다.
- 이진 탐색 트리는 이러한 이진 탐색의 알고리즘이 이진 트리에 적용된 형태로 트리의 루트노드는 이진 탐색에서 리스트의 중간값이 된다.
 - 루트노드의 왼편 서브 트리의 값들은 이진 탐색 알고리즘에 기반하여 모두 루트노드의 값보다 작은 값들이 자리 잡고 있고 루트노드의 오른편 서브 트리의 값들은 루트노드의 값보다 큰 값들이 자리 잡고 있어야 한다.
 - $Left < root < Right$

정리하자면 다음과 같다.

각 노드에 중복되지 않는 키(Key)가 있다.루트노드의 왼쪽 서브 트리는 해당 노드의 키보다 작은 키를 갖는 노드들로 이루어져 있다.루트노드의 오른쪽 서브 트리는 해당 노드의 키보다 큰 키를 갖는 노드들로 이루어져 있다.좌우 서브 트리도 모두 이진 탐색 트리여야 한다.

즉 이진 탐색 트리(Binary Search Tree)는 모든 왼쪽 자식의 값이 루트나 부모보다 작고, 모든 오른쪽 자식의 값이 루트나 부모보다 큰 값을 가지는 특징이 있다.



이진 탐색 트리(Binary Search Tree)

이진탐색트리 insert(삽입) 코드 - while문 활용

```
#include <iostream>
#include <vector>
#include <list>
#include <string>
#include <algorithm>
#include <map>

int bst[256] = { 0, };

void insert(int data)
{
```

```

int idx = 1;
while (1)
{
    if (bst[idx] == 0)
    {
        bst[idx] = data;
        break;
    }
    else if (bst[idx] > data)
    {
        idx = idx * 2;
    }
    else
    {
        idx = idx * 2 + 1;
    }
}

int main()
{
    insert(10);
    insert(5);
    insert(15);
    //insert(8);
    //insert(3);
    //insert(7);
    //insert(12);

    return 0;
}

```

이진탐색트리 insert(삽입) 코드 - 재귀함수 활용

```

#include <iostream>
#include <vector>
#include <list>
#include <string>
#include <algorithm>
#include <map>

int bst[256] = { 0, };

void insertRecursive(int data, int now)
{
    if (bst[now] == 0)
    {
        bst[now] = data;
        return;
    }
    if (bst[now] > data)
    {
        insertRecursive(data, now * 2);
    }
    else
    {
        insertRecursive(data, now * 2 + 1);
    }
}

int main()
{
    insertRecursive(10, 1);
    insertRecursive(5, 1);
    insertRecursive(15, 1);
    insertRecursive(8, 1);

    return 0;
}

```

이진탐색트리 Search(탐색) 코드 - while문, 재귀함수 활

```
#include <iostream>
#include <vector>
#include <list>
#include <string>
#include <algorithm>
#include <map>

int bst[256] = { 0, };

void insert(int data)
{
    int idx = 1;
    while (1)
    {
        if (bst[idx] == 0)
        {
            bst[idx] = data;
            break;
        }
        else if (bst[idx] > data)
        {
            idx = idx * 2;
        }
        else
        {
            idx = idx * 2 + 1;
        }
    }
}

void insertRecursive(int data, int now)
{
    if (bst[now] == 0)
```

```

{
    bst[now] = data;
    return;
}
if (bst[now] > data)
{
    insertRecursive(data, now * 2);
}
else
{
    insertRecursive(data, now * 2 + 1);
}
}

void search(int data)
{
    int idx = 1;
    while (1)
    {
        if (bst[idx] == 0)
        {
            printf("Not Found\n");
            break;
        }
        if (bst[idx] == data)
        {
            printf("Found\n");
            break;
        }
        if (bst[idx] > data)
        {
            idx = idx * 2;
        }
        else
        {
            idx = idx * 2 + 1;
        }
    }
}

```

```

}

void searchRecursive(int data, int now)
{
    if (bst[now] == 0)
    {
        printf("Not Found\n");
        return;
    }
    if (bst[now] == data)
    {
        printf("Found\n");
        return;
    }
    if (bst[now] > data)
    {
        searchRecursive(data, now * 2);
    }
    else
    {
        searchRecursive(data, now * 2 + 1);
    }
}

```

```

int main()
{

    // Binary Search Tree
    // 데이터를 탐색하는데 있어서 효율성 높다.
    // 3가지 동작을 기본으로 둔다.
    // 데이터를 저장하는 Insert
    // 데이터를 찾는 Search
    // 데이터를 삭제하는 Delete
    // insert, search 위조루 보겠다.
    // 사용하는 제일 큰 이유는 데이터를 더 빨리 찾기위해 사용하는 자료구조이다.

    //insert(10);

```



```

//insert(5);
//insert(15);
//insert(8);
//insert(3);
//insert(7);
//insert(12);

insertRescursive(3, 1);
insertRescursive(5, 1);
insertRescursive(1, 1);
insertRescursive(2, 1);
insertRescursive(4, 1);
insertRescursive(7, 1);

return 0;
}

```

유니온 파인드 알고리즘이란?



유니온 파인드 알고리즘: 상호 배타적 집합, Disjoin-set(서로소 집합) 이라고도 부른다. 여러 노드가 존재할 때 어떤 두 개의 노드를 같은 집합으로 묶어 주고, 어떤 두 노드가 같은 집합에 있는지 확인하는 알고리즘이다.

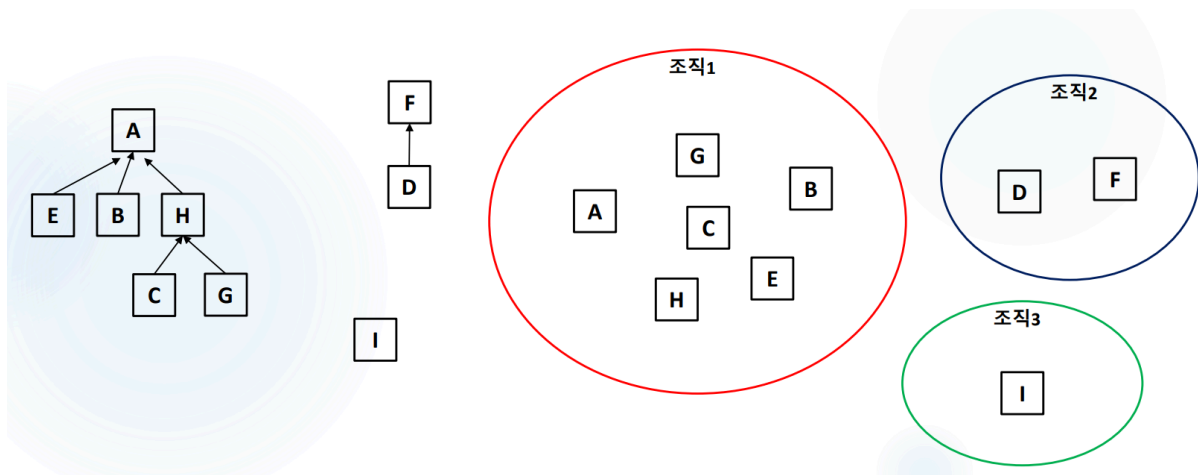
왜 이름이 유니온 파인드인가?

유니온 파인드 알고리즘은 두가지의 연산을 진행한다. 이 연산 과정을 Union, Find라고 부른다.

- **Union:** 서로 다른 두 개의 집합을 하나의 집합으로 병합하는 연산을 말한다. 이 자료구조에서는 상호 배타적 집합만을 다루므로 Union 연산은 합집한 연산과 같다.
- **Find:** 하나인 원소가 어떤 집합에 속해있는지를 판단한다.

크루스칼 알고리즘과 프림 알고리즘을 알기 위해선 유니온 파인드 알고리즘을 알아야 한다.

아래 그림은 각 데이터들을 총 3그룹으로 묶었다.



배열을 이용한 UnionFind Insert 구현해보기

```
#include <iostream>
#include <vector>
#include <list>
#include <string>
#include <algorithm>
#include <map>

int name[256] = {};
int n = 0;

int group[256] = {};
int gCnt = 0;

void insert(char a, char b)
{
    if (group[a] == 0) name[n++] = a;
    if (group[b] == 0) name[n++] = b;

    // a는 그룹에 있는데, b는 그룹에 없다.
    if (group[a] != 0 && group[b] == 0)
        group[b] = group[a];
    // a그룹은 없고, b 그룹은 있다.
```

```

else if (group[a] == 0 && group[b] != 0)
    group[a] = group[b];
//둘다 그룹이 없는경우는 새로운그룹을 만든다.
else if (group[a] == 0 && group[b] == 0)
{
    gCnt++;
    group[a] = gCnt;
    group[b] = gCnt;
}
else
{
    // 둘다 그룹이 있는 경우, 그룹을 a로 통합시킨다.
    int g = group[b];
    for (int i = 0; i < n; i++)
    {
        if (group[name[i]] == g)
            group[name[i]] = group[a];
    }
}
}

int main()
{
    // 그룹으로 관리하는것은 분류하거나 검색하는데 있어서 유용하다.
    // 그룹끼리 통합을 시키면 큰데이터를 연산하지 않고 작은 데이터로 연산할 수 있다.

    // 그룹 1
    insert('A', 'B');
    insert('A', 'C');

    // 그룹2
    insert('E', 'Q');
    insert('E', 'F');

    // 통합
    insert('F', 'A');

```

```
    return 0;
}
```

재귀함수를 이용하여 UnionFind Insert구현해보기

```
#include <iostream>
#include <vector>
#include <list>
#include <string>
#include <algorithm>
#include <map>

char parnet[1000];

char getParent(char x)
{
    if (parnet[x] == 0)
        return x;

    int ret = getParent(parnet[x]);
    parnet[x] = ret;

    return ret;
}

void insert(char ch1, char ch2)
{
    int a = getParent(ch1);
    int b = getParent(ch2);

    if (a != b)
        parnet[b] = a;
}

int main()
```

```
{
    // 그룹으로 관리하는것은 분류하거나 검색하는데 있어서 유용하다.
    // 그룹끼리 통합을 시키면 큰데이터를 연산하지 않고 작은 데이터로 연산할 수 있다.

    insert('A', 'G');
    insert('H', 'C');
    insert('A', 'H');
    insert('F', 'D');
    insert('A', 'F');

    return 0;
}
```

"강의는 많은데, 왜 나는 아직도 코드를 못 짤까?"

혼자 공부하다 보면 누구나 이런 고민을 하게 됩니다.

- 강의는 다 들었지만 막상 **손이 안 움직이고**,
- 복습을 하려 해도 **무엇을 다시 봐야 할지 모르겠고**,
- 질문할 곳도 없고,
- 유튜브는 결국 **정답을 따라 치는 것밖에 안 되는 것 같고**.

문제는 **'연습'이 빠졌기 때문**입니다.

단순히 강의를 듣는 것만으로는 실력이 늘지 않습니다.

실제 문제를 풀고, 고민하고, 직접 구현해보는 시간이 반드시 필요합니다.

그래서, **얌얌코딩 코칭**은 다릅니다.

그냥 가르치지 않습니다.

스스로 설계하고, 코딩할 수 있게 만듭니다.

얌얌코딩 코칭에서는 단순한 예제가 아닌,

스스로 문제를 분석하고 구현해야 하는 연습문제를 제공합니다.

이 연습문제들은 다음과 같은 역량을 키우기 위해 설계되어 있습니다:

- 문제를 스스로 쪼개고 설계하는 힘
- 다양한 조건을 만족시키는 실제 구현 능력
- 기능 단위가 아닌, 프로그램 단위로 사고하는 습관
- 마침내 자신의 힘으로 코드를 끝까지 작성하는 경험

지금 필요한 건 더 많은 강의가 아닙니다.

코드를 스스로 완성해 나가는 훈련,

그것이 지금 실력을 끌어올릴 가장 현실적인 방법입니다.

👉 자세한 안내 보기: [프리미엄 코칭 안내 바로가기](#)

👉 또는 카카오톡 상담방: [얌얌코딩 상담방](#)

▼ oopy:hide



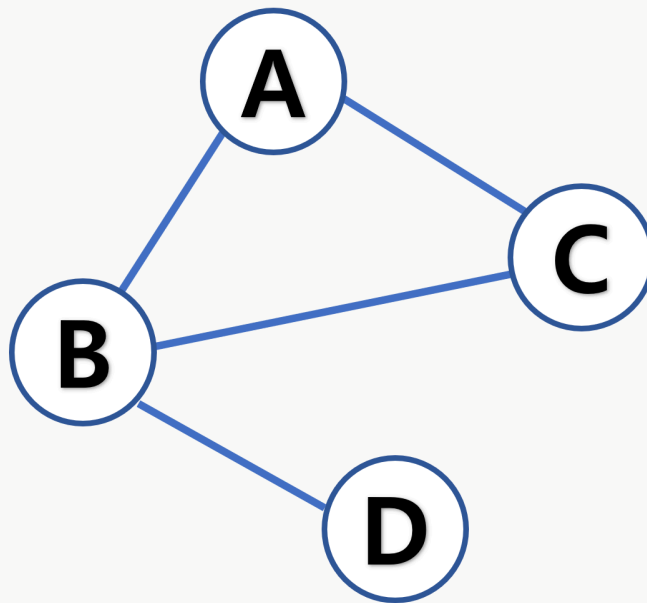
문제 1번

노드 4개의 연결상태를 입력 받으세요.

Cycle이 있는 그래프인지 아닌지 판별하면 됩니다.

Cycle을 확인 후 발견 또는 미발견을 출력 해주세요.

만약, 다음과 같은 그래프를 입력 받았다면 Cycle이 있으므로 "발견"을 출력 하면 됩니다.



[힌트]

union-find로 같은 그룹인지 아닌지를 알아내어

Cycle의 여부를 판단할 수 있습니다.

입력 예제

4

A C

A B

B D

C B

출력 결과

발견

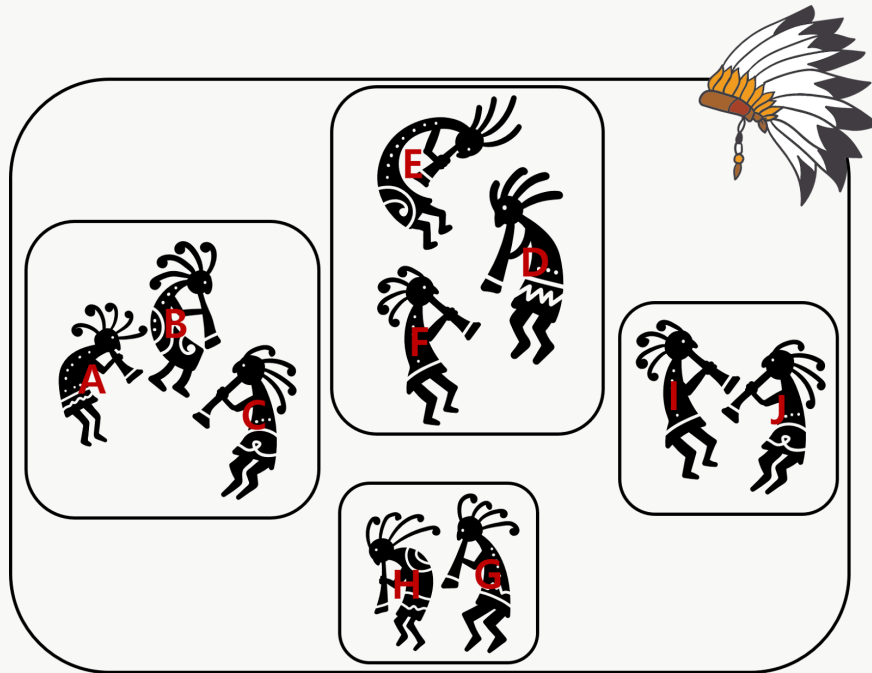


문제 2번

원시족을 나타내는 각 문자들이 아래와 같이 부족을 형성하고 있습니다.

아래 그림은 부족의 초기 상태를 나타냅니다.

아래와 같이 같이 그룹이 지어진 상태에서 시작합니다.



위 그림과 같은 초기 상태에서 시작합니다.

따라서, 입력을 받기 전 남아있는 그룹의 수는 총 4개 입니다.

이제 그룹으로 합쳐질 n개의 문자 쌍을 입력 받아주세요.

n개의 문자 쌍을 모두 Grouping을 했을 때, 최종적으로 몇 개의 그룹이 남아있는지 출력 하세요.

[예시]

3

G I // G와 I가 속한 그룹을 합침 (현재 그룹수 : 4 --> 3개)

D E // D와 E가 속한 그룹을 합침, 하지만 이미 같은 그룹이라 그룹의 수는 변동 없음 (현재 그룹 수 : 3개)

H J // D와 E가 속한 그룹을 합침, 하지만 이미 같은 그룹이라 그룹의 수는 변동 없음 (현재 그룹 수 : 3개)

출력결과 : 3개

입력 예제

3

G I

D E

H J

출력 결과

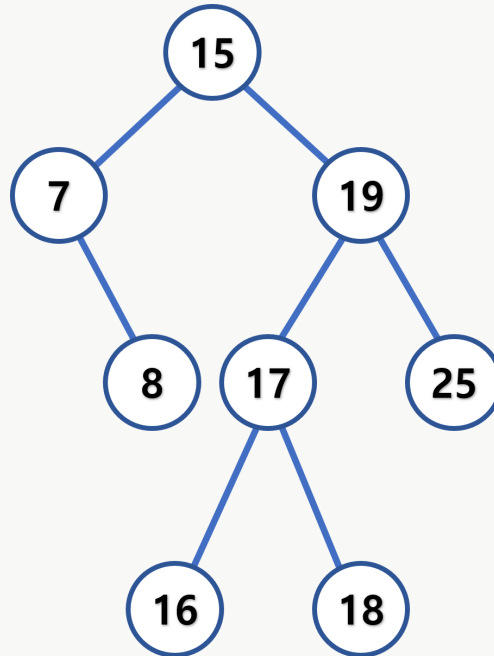
3개



문제 3번

숫자 5개를 5칸짜리 target 배열에 입력 받아주세요.

그리고 다음 Binary Search Tree를 1차원 배열에 하드코딩 해 주세요



재귀호출을 이용해서

target 숫자들이 각각 존재하는지 찾아서 출력 해 주세요

(찾는 find함수를 while로 구현하시면 됩니다)

입력 예제

8 16 9 25 30

출력 결과

8:3회만에찾음

16:4회만에찾음

9:없음

25:3회만에찾음

30:없음

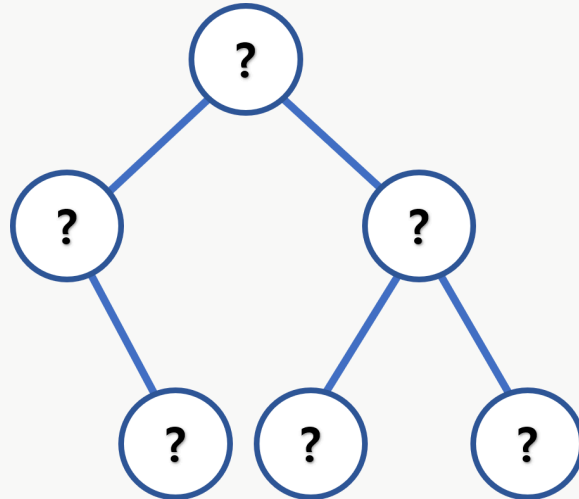


문제 4번

BST 자료구조에 값을 넣을 n개의 숫자를 입력받으세요.

이제 1 ~ 6 까지 각 숫자들이 BST에 존재하는지 출력하면 됩니다.

(insert함수와 find함수는 모두 재귀호출로 구현 해 주세요.)



[참고]

BST는 Binary Search Tree 자료구조입니다.

Binary Search 알고리즘와는 다릅니다.

(인터뷰 볼때 혼용하지 마세요, Binary Search는 다음 시간에 배우게 됩니다)

입력 예제

4

1 3 4 7

출력 결과

1:O

2:X

3:O

4:O

5:X

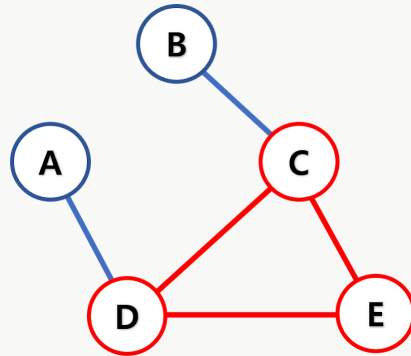
6:X



문제 5번

노드의 개수와 인접행렬을 입력받고, cycle이 존재하는지 출력하세요.

[예시 1]

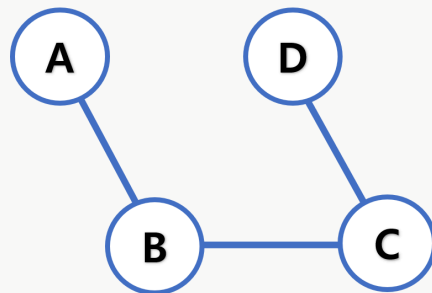


	A	B	C	D	E
A	0	0	0	1	0
B	0	0	1	0	0
C	0	0	0	1	1
D	1	0	1	0	1
E	0	0	1	1	0

위 그래프는 cycle이 존재합니다. (C-D-E)

출력결과 : cycle 발견

[예시 2]



	A	B	C	D
A	0	1	0	0
B	1	0	1	0
C	0	1	0	1
D	0	0	1	0

위 그래프는 cycle이 없습니다.

출력결과 : 미발견

[힌트]

간선을 발견할때마다, Grouping을 하면 됩니다.

만약 같은 그룹이 발견되면 Cycle 입니다.

입력 예제

5

0 0 0 1 0

0 0 1 0 0

0 0 0 1 1

1 0 1 0 1

0 0 1 1 0

출력 결과

cycle 발견

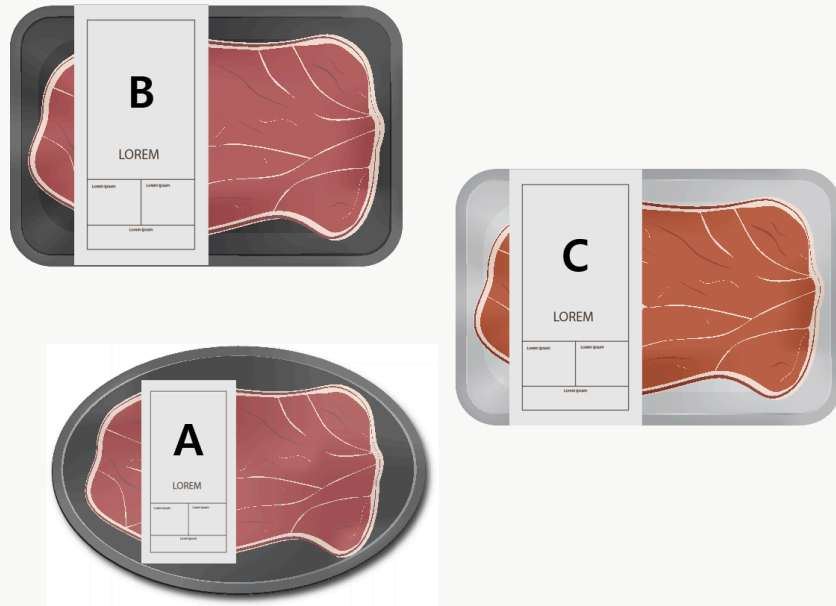


문제 6번

Minco 스테이크 레스토랑에서는 매일 고기 재료를 납품 받습니다.

1 ~ K번의 품목 번호를 가진 고기들은 A ~ F까지 각각의 등급을 가지고 있습니다.

직원들은 어떤 품목들이 들어왔는지 품목리스트에 적어 둡니다.



고기의 납품 이력이 적혀있는 품목 리스트의 내용들을 적절한 자료구조에 저장 해 주세요.

그리고 각 번호의 고기들이 각각 몇 등급인지 순차적으로 출력 해주는 프로그램을 제작 해 주세요.

예를들어, 품목 리스트에 다음과 같이 적혀있다고 가정합니다.

"3 2" // 3 번과 2 번 고기는 같은 등급입니다.

"B 4" // 4 번 고기는 B 등급 입니다.

"1 A" // 1 번 고기는 A 등급 입니다.

"3 4" // 3 번과 4 번 고기는 같은 등급 입니다.

이때 2, 3, 4번 고기 등급은 B 입니다.

[입력]

품목 리스트의 개수 N, 레스토랑에서 받은 고기 개수 K를 먼저 입력 받습니다.

그리고 N개의 품목 리스트를 입력 받습니다.

품목리스트는 숫자 2 개 or 1개 문자 + 1개 숫자로 구성되어 있습니다.

$1 \leq N \leq 20$

$1 \leq K \leq 9$

[출력]

1번부터 K번까지 각각 몇 등급인지 순서대로 출력하면 됩니다.

품목리스트를 확인했을 때, 무슨 등급인지 알 수 없는 입력데이터는 주어지지 않습니다.

입력 예제

6 4

3 2

B 4

1 A

3 4

3 B

A 1

출력 결과

ABBB



문제 7번

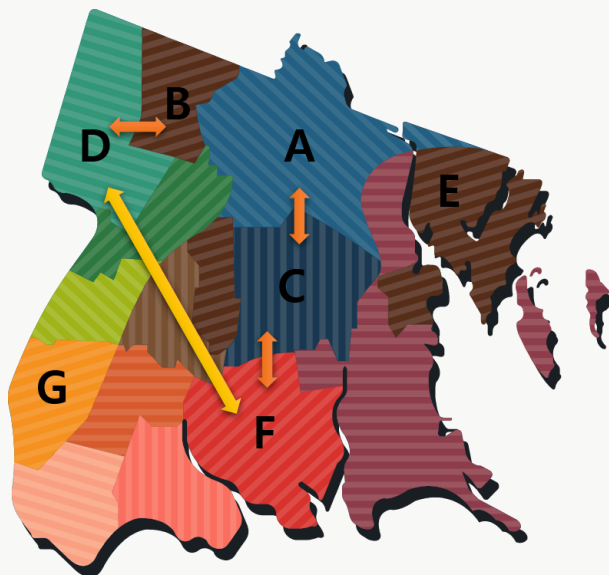
춘추 전국 시대에는 국가끼리 동맹을 맺고, 서로 침략하고, 전쟁을 일으키곤 했습니다.

이 시대에는 N개의 국가가 존재하며, 각 국가들은 인구수를 갖고 있습니다.
아래 그림은 총 7개의 나라를 나타낸 지도이며,
D와 B는 동맹을 맺고, A와 C와 F가 동맹을 맺었습니다.



이러한 상황에서, D와 F가 전쟁을 일으켰습니다.

만약 $A+C+F$ 동맹의 총 인구수가 $D+B$ 동맹의 총 인구수보다 더 크다면,
 $A+C+F$ 동맹의 승리로 D 국가와 B 국가는 멸망하고 맙니다.



먼저 각 국가의 수와 각 국가의 인구수를 입력 받습니다.

그리고 동맹을 맺고 (alliance 명령어), 전쟁을 일으키는 (war 명령어) 상황을 입력받고,

살아남은 국가의 수를 출력 해 주세요.

[예시]

7 // 7개의 국가(N)

10 20 30 40 50 60 70 // 각 국가의 인구수

3 // 3개의 상황

alliance A C // A와 C 동맹

alliance F C // F와 C 동맹

alliance D B // D와 B 동맹

alliance A F // A와 F 동맹 but, 이미 동맹이 되어있으므로 무시

war D F // D 동맹국들과 F 동맹국들의 전쟁

A + C + F 동맹국의 인구수가 더 많으므로, D와 B 국가는 멸망합니다.

따라서 총 5 개의 국가가 살아 남았으므로 출력결과는 5 입니다.

[세부조건]

1. 같은 동맹끼리 전쟁을 일으키지 않습니다.

2. 동맹을 파기하는 일은 없습니다.

입력 예제

7

10 20 30 40 50 60 70

5

alliance A C

alliance F C

alliance D B

alliance A F

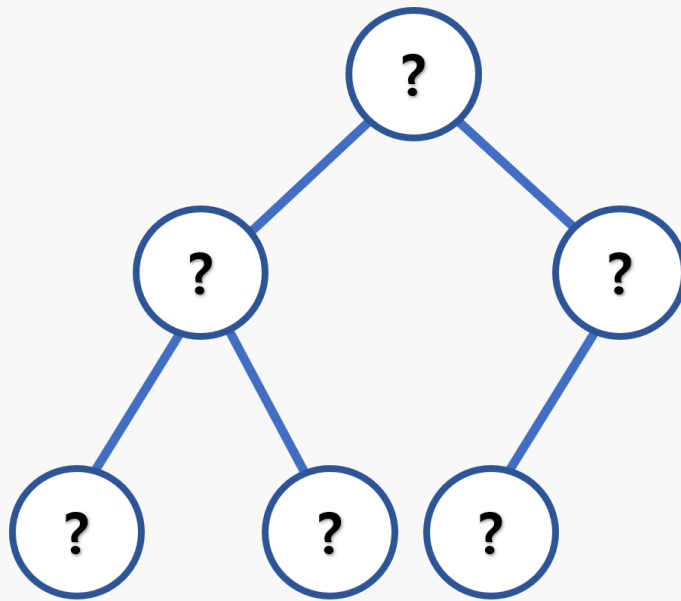
war D F

출력 결과



문제 8번

5개의 숫자를 Binary Search Tree 자료구조에 저장 해 주세요.
작거나 같은 숫자는 왼쪽, 큰 숫자는 오른쪽에 저장하는 BST에서
DFS를 이용하면 쉽게 숫자를 정렬하여 출력할 수 있습니다.



BST를 자료구조를 만들고,
5개의 숫자를 입력받아 BST에 순서대로 insert 해 주세요.
만들어진 이진트리를 탐색하여, 정렬 결과를 출력해 주세요.

입력 예제

5 7 2 3 1

출력 결과

1 2 3 5 7